INTERNSHIP/PROJECT REPORT

Submitted By

Doshi Zalak Manishbhai

(*216090307005*)

To

COMPUTER ENGINEERING DEPARTMENT

C U SHAH GOVT. POLYTECHNIC – SURENDRANAGAR



GUJARAT TECHNOLOGICAL UNIVERSITY – AHMEDABAD

JULY-AUGUST – 2023

INDEX

Sr No	Content
1	Student Registration Form
2	Attendance Sheet
3	Daily Log with Topic Learned (Day or Week wise)
4	Feedback from Industry
5	Completion Certificate from Industry
6	Certificate from Institute
7	Project Introduction
8	Frontend and Back-end Technologies of project
9	Functional and Non-Functional requirements of project
10	List of Users and Use case Diagrams for each user of project
11	Data Dictionary of project
12	Sample coding of project
13	Screen shot of various pages of project
14	Testing strategies of project
15	Limitations of Project
16	Conclusion and Future work
17	References

Project Introduction

The E-Commerce Clothing Website is a cutting-edge online platform that revolutionizes the way people shop for fashion. Our project is driven by a mission to deliver an exceptional and hassle-free shopping experience to fashion enthusiasts of all ages. Here's a more detailed introduction:

Innovative Shopping Experience: Our website is not just another online clothing store; it's a fashion destination that brings innovation to your fingertips. We've combined the latest technologies with a user-centric design to create a shopping experience that's both enjoyable and efficient.

User-Friendly Design: Navigating our website is a breeze. Our user-friendly interface ensures that you can effortlessly explore our collections, discover new styles, and find the perfect outfit for any occasion.

Secure Shopping: Your security is our priority. We've implemented robust security measures to protect your personal and financial information, allowing you to shop with confidence.

Wish list Functionality: Not sure about a purchase just yet? Use our wishlist feature to save your favourite items for later. It's a convenient way to keep track of what caught your eye.

Diverse Categories: Our products are organized into various categories, making it easy to find what you're looking for. Whether it's casual wear, formal attire, accessories, or footwear, you'll find it in our well-defined sections.

Constant Updates: Fashion evolves, and so do we. We regularly update our collections to keep up with the latest trends, ensuring that you stay at the forefront of fashion.

Responsive Support: Need assistance? Our customer support team is always ready to help. We're just a message away, ensuring that your shopping journey is smooth and enjoyable.

The E-Commerce Clothing Website is not just a place to shop; it's a fashion experience tailored to your needs. We invite you to explore our platform, discover your style, and embark on a fashion journey like never before. Welcome to the future of online shopping!

Frontend & Backend technologies of project

The project utilizes a combination of technologies for both its frontend and backend development. The project is basically build using MERN. Let's introduce the technologies used in each:

➢ Frontend Technologies:

1. **React:** The frontend of the E-Commerce Clothing Website is built using React, a popular JavaScript library for building user interfaces. React allows for the creation of dynamic and responsive web pages.

2. **Redux:** Redux is used for state management in the frontend. It provides a predictable state container and helps manage the application's global state.

3. **Axios:** Axios is used for making HTTP requests from the frontend to the backend. It facilitates the retrieval and posting of data between the client and server.

4. **HTML/CSS:** HTML is used for structuring web pages, while CSS is employed for styling and layout. Together, they ensure an appealing and user-friendly interface.

➢ Backend Technologies:

1. **Node.js:** The backend of the project is powered by Node.js, which allows server-side scripting using JavaScript. Node.js is known for its speed and scalability, making it ideal for building the server-side logic of web applications.

2. **Express:** Express is a web application framework for Node.js. It simplifies the creation of APIs and routes, making it easier to handle HTTP requests and responses.

3. **MongoDB:** MongoDB serves as the project's database. It's a NoSQL database that stores data in a flexible, JSON-like format. MongoDB is well-suited for handling large volumes of unstructured data.

4. **Mongoose:** Mongoose is an ODM (Object Data Modeling) library for MongoDB. It simplifies interaction with the database, providing a schema-based solution for managing data.

5. **Bcrypt:** Bcrypt is used for hashing user passwords, ensuring the security of user data.

6. **Cors:** Cors is a middleware used to enable cross-origin resource sharing. It permits the frontend to make requests to the backend from a different domain.

These technologies collectively power the E-Commerce Clothing Website, enabling seamless communication between the frontend and backend, efficient data storage and retrieval, and a responsive user interface.

Functional & Non-Functional requirements of project

➢ Functional Requirements:

1. **User Registration and Authentication:-**

- Users must be able to create accounts with unique usernames and valid email addresses.
- Passwords should be securely stored and hashed.
- Users must be authenticated to access certain features like making purchases, viewing the wish list, and leaving reviews.

2. **Product Listings:-**

- Products should be categorized into sections like all, men's clothing, women's clothing, etc.
- Each product listing should include details such as name, description, price, available quantities, and images.

3. **Search and Filters:-**

- Users should be able to search for products using keywords i.e basically through product's name.
- Filters by category, name and brand should be available to refine search results.

4. **Shopping Cart:-**

- Users should be able to add products to their shopping carts.
- The cart should display the selected items, their quantities, and the total cost.
- Users can modify the cart by adding/removing items.

5. **Wishlist:-**

- Users can add products to their wishlist for future reference.
- Users can modify the wishlist by adding/removing items.
- The wishlist should display the selected items, their quantities, and the total cost.

6. **Product Reviews:-**

- Registered users can leave reviews for products.

- Reviews should include text.

7. Contact and Support:-

- Users can contact customer support through a contact form and the owner immediately gets the email with customer's name, email and review.
- Responses to inquiries should be timely.

➢ Non-Functional Requirements:

1. Performance:-

- The website should load quickly, even with a large number of products and users.
- It should handle concurrent user sessions efficiently.

2. Scalability:-

- The system should be scalable to accommodate a growing user base and product catalog.

3. Security:-

- User data, especially passwords, should be securely stored and transmitted.
- Payment processing must comply with industry security standards (e.g., PCI DSS).

4. Usability:-

- The user interface should be intuitive and accessible on various devices (desktop, mobile, tablet).
- Users with disabilities should have an accessible experience.

5. Availability:-

- The website should have high availability, with minimal downtime for maintenance.
- Regular backups of data should be performed.

6. Reliability:-

- The system should be reliable, with minimal errors, crashes, or data corruption.

7. **Data Privacy and Compliance:-**

- The system should adhere to data protection laws and regulations (e.g., GDPR).
- User consent for data collection and processing should be obtained when required.

8. **Testing:-**

- Thorough testing, including unit tests, integration tests, and user acceptance testing, should be conducted before and after deployment.

9. **Documentation:-**

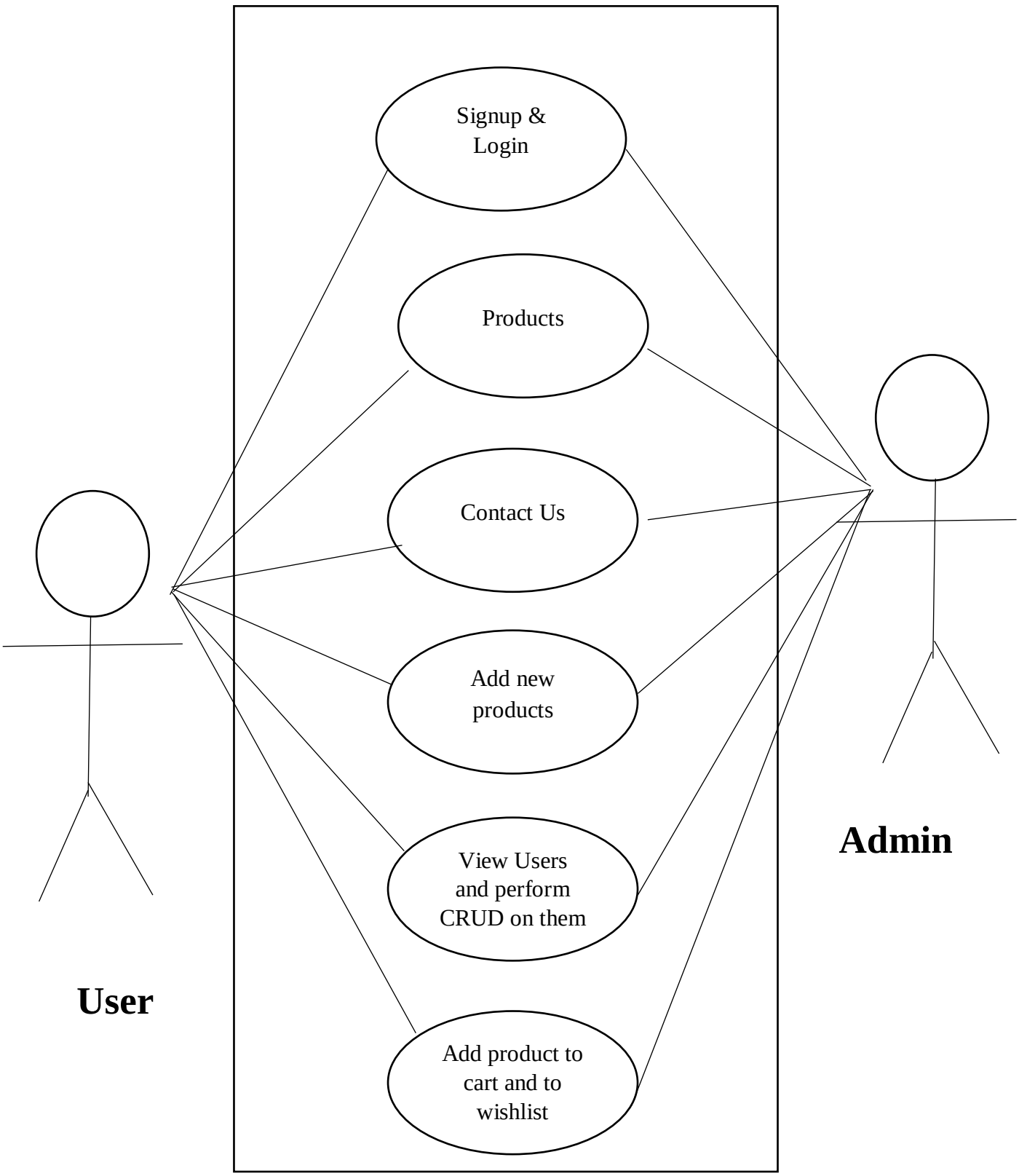
- Comprehensive documentation for users, administrators, and developers should be available.

10. **Load Handling:-**

- The system should be able to handle high traffic during sales events or promotions.

These functional and non-functional requirements serve as a foundation for developing a robust and user-friendly e-commerce clothing website. They ensure that the platform meets both user expectations and industry standards.

List of Users and Use case Diagrams for each user of project



Data Dictionary of project

Table: Users

Field Name	Data Type	Description
UserID	intint	Unique for a user
Username	Varchar(50)	Name of the user
Email	Varchar(100)	Email of the user
Password	Varchar(50)	Password of the user
Contact	Number(10)	Number of the user
Type	Varchar(20)	Type of the user

Table: Product

Field Name	Data Type	Description
ProductId	int	Unique for a user
Name	Varchar(100)	Name of the product
Image	Binary	Image of the product
Description	text	Detailed description of the product
Category	Varchar(20)	Category of the product
Price	Number	Price of the product
Quantity	Number	Quantity of the product

Table: Categories

Field Name	Data Type	Description
CategoryId	int	Unique identifier for a category
Name	Varchar(50)	Name of the category

Table: CartItem

Field Name	Data Type	Description
Image	Binary	Image of the product added to cart
Name	Varchar(100)	Name of the product added to cart
Price	int	Price of the product added to cart
Quantity	int	Quantity of the product added to cart
Action	button	Product you want to remove from cart
Total Products	int	Total numbers of the product in the cart
Total Price	number	Total number of the price of the product added to cart

Table: WishlistItem

Field Name	Data Type	Description
Image	Binary	Image of the product added to cart
Name	Varchar(100)	Name of the product added to cart
Price	int	Price of the product added to cart
Action	button	Product you want to remove from cart

Table: Contact

Field Name	Data Type	Description
Name	Varchar(200)	Name of the user
Email	Varchar(200)	Email of the user
Your Review or Message	Varchar(200)	Your message or review you want to give to the admin

Backend:-

User-controller.js:-

```
import User from "../models/Users"; // Use the correct casing here
import bcrypt from "bcryptjs";

export const getAllUser = async (req, res, next) => {
  let users;
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;

  const startIndex = (page - 1) * limit;
  const endIndex = page * limit;

  try {
    users = await User.find().skip(startIndex).limit(limit);
    if (!users) {
      return res.status(200).json({ message: "No User Found" });
    } else {
      return res.status(200).json({
        totalCount: users.length,
        currentPage: page,
        totalPages: Math.ceil(users.length / limit),
        User: users,
      });
    }
  } catch (err) {}
  console.log(err);
};
```

```
export const show = async (req, res, next) => {
  let users;
  const id = req.params.id;

  try {
    users = await User.findById(id);
  } catch (err) {
    console.log(err);
  }
  if (!users) {
    return res.status(200).json({ message: "No User Found" });
  } else {
    return res.status(200).json({ users });
  }
};

export const destroy = async (req, res, next) => {
  let users;
  const id = req.params.id;
  const status = "Delete";
  try {
    users = await User.findByIdAndUpdate(id, { status: status });
  } catch (err) {
    console.log(err);
  }
  if (!users) {
    return res.status(200).json({ message: "No User Found" });
  } else {
    return res.status(200).json({ users });
  }
};

export const update = async (req, res, next) => {
  let users;
```

```
  const id = req.params.id;
  const { username, email, contact, type, status } = req.body;
  if (!username) {
    res.status(200).json({ message: "Enter name" });
  } else if (!email) {
    res.status(200).json({ message: "Enter email" });
  } else if (!contact) {
    res.status(400).json({ message: "Enter Contact" });
  } else if (!type) {
    res.status(400).json({ message: "Enter Type" });
  } else {
    try {
      users = await User.findByIdAndUpdate(id, {
        username: username,
        email,
        contact: contact,
        type: type,
        status: status,
      });
      if (!User) {
        return res.status(200).json({ message: "No User Found" });
      } else {
        return res.status(200).json({ users });
      }
    } catch (err) {
      console.log(err);
    }
  }
};

export const singup = async (req, res, next) => {
  const { username, email, password, contact, type, status } = req.body;

  let existingUser;
```

```
try {
  existingUser = await User.findOne({ email });
} catch (err) {
  return console.log(err);
}
if (existingUser) {
  return res
    .status(400)
    .json({ message: "User Already Exists! Login instead" });
}
const hashedPassword = bcrypt.hashSync(password);
if (!username) {
  res.status(200).json({ message: "Enter name" });
} else if (!email) {
  res.status(200).json({ message: "Enter email" });
} else if (!password) {
  res.status(200).json({ message: "Enter Password" });
} else if (!contact) {
  res.status(200).json({ message: "Enter Contact" });
} else if (!type) {
  res.status(200).json({ message: "Enter Type" });
} else {
  const user = new User({
    username,
    email,
    password: hashedPassword,
    contact,
    type,
    status: "Active",
  });

  try {
    await user.save();
  } catch (err) {
```

```
      return console.log(err);
    }
    return res.status(201).json({ user });
  }
};

export const login = async (req, res, next) => {
  const { email, password } = req.body;
  let existingUser;
  try {
    existingUser = await User.findOne({ email });
  } catch (err) {
    return console.log(err);
  }
  if (!existingUser) {
    return res
      .status(404)
      .json({ message: "Couldn't find user by this Email" });
  }

  const isPasswordCorrect = bcrypt.compareSync(password, existingUser.password);
  if (!isPasswordCorrect) {
    return res.status(400).json({ message: "Incorrect Password" });
  }
  return res.status(200).json({ message: "Login Successfull" });
};
```

User-route.js:-

```
import express from "express";
import {
  getAllUser,
  singup,
  login,
  show,
```

```
  destroy,
  update,
} from "../controllers/user-controller";

const router = express.Router();

router.get("/", getAllUser);
router.get("/show/:id", show);
router.delete("/delete/:id", destroy);
router.post("/singup", singup);
router.post("/update/:id", update);
router.post("/login", login);
export default router;
```

users.js

```
import Express from "express";
import mongoose from "mongoose";

const Schema = mongoose.Schema;
const userSchema = new Schema({
  username: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    required: true,
    unique: true,
  },
  password: {
    type: String,
    required: true,
    minlength: 6,
  },
```

```
  contact: {
    type: String,
    required: true,
    minlength: 10,
  },
  type: {
    type: String,
  },
  status: {
    type: String,
  },
});
export default mongoose.model("User", userSchema);
```

index.js:-

```
import express from "express";
import cors from "cors";
import mongoose from "mongoose";
import router from "./routes/user-route";

const app = express();
app.use(express.json());
app.use(cors());
app.use(express.static("public"));
app.use("/backend/user", router);

mongoose
  .connect(

"mongodb+srv://zalakdoshi2005:zalakdoshi2005@cluster0.rnl5qhl.mongodb.net/Intern_project?retryWrites=true&w=majority"
  )
  .then(() => app.listen(5000))
  .then(() =>
```

```
  console.log("Conected To Database And Listening To Localhost 5000")
 )
 .catch((err) => console.log(err));
```

Frontend:-

App.js:-

```
import React, { useState } from "react";
import {
 BrowserRouter as Router,
 Routes,
 Route,
 NavLink,
} from "react-router-dom";
import Home from "./Home";
import Navbar from "./Navbar";
import Signup from "./Signup";
import Login from "./Login";
import About from "./About";
import Contact from "./Contact";
import Products from "./Products";
import Cart from "./Cart";
import Wishlist from "./Wishlist";

function App() {
 const [isLoggedIn, setIsLoggedIn] = useState(false);
 const [username, setUsername] = useState("");
 const [cartItems, setCartItems] = useState([]);
 const [wishlistItems, setWishlistItems] = useState([]); // Declare wishlistItems here

 const handleLogin = (username) => {
```

```
  setUsername(username);
  setIsLoggedIn(true);
};

const handleAddToCart = (product) => {
  setCartItems([...cartItems, product]);
};

const handleRemoveFromCart = (productToRemove) => {
  const updatedCartItems = cartItems.filter(
    (product) => product.id !== productToRemove.id
  );
  setCartItems(updatedCartItems);
};

const handleUpdateQuantity = (updatedProduct, newQuantity) => {
  const updatedCartItems = cartItems.map((item) => {
    if (item.id === updatedProduct.id) {
      return { ...item, quantity: newQuantity };
    }
    return item;
  });
  setCartItems(updatedCartItems);
};

const handleLogout = () => {
  setUsername("");
  setIsLoggedIn(false);
};

// Define handleAddToWishlist to add products to the wishlist
const handleAddToWishlist = (product) => {
  setWishlistItems([...wishlistItems, product]);
};
```

```
// Placeholder for handleRemoveFromWishlist
const handleRemoveFromWishlist = (productToRemove) => {
  // Implement your logic for removing from the wishlist here
  const updatedWishlistItems = wishlistItems.filter(
    (product) => product.id !== productToRemove.id
  );
  setWishlistItems(updatedWishlistItems);
};

return (
  <Router>
    <Navbar
      isLoggedIn={isLoggedIn}
      username={username}
      onLogout={handleLogout}
      cartItems={cartItems}
      wishlistItems={wishlistItems} // Pass wishlistItems here
    >
      {!isLoggedIn ? (
        <>
          <NavLink exact to="/login" activeClassName="active">
            Login
          </NavLink>
          <NavLink exact to="/signup" activeClassName="active">
            Signup
          </NavLink>
        </>
      ) : (
        <>
          <NavLink exact to="/" activeClassName="active">
            Home
          </NavLink>
        </>
```

```
 )}
</Navbar>

<div className="content">
 <Routes>
  <Route
   path="/"
   element={isLoggedIn ? <Home /> : <Home onLogin={handleLogin} />}
  />
  <Route path="/about" element={<About />} />
  <Route
   path="/products"
   element={
    isLoggedIn ? (
     <Products
      onAddToCart={handleAddToCart}
      onAddToWishlist={handleAddToWishlist} // Pass wishlist handler
     />
    ) : (
     <Signup onLogin={handleLogin} />
    )
   }
  />
  <Route path="/contact" element={<Contact />} />
  <Route
   path="/cart"
   element={
    <Cart
     cartItems={cartItems}
     onRemoveFromCart={handleRemoveFromCart}
     onUpdateQuantity={handleUpdateQuantity}
    />
   }
  />
```

```
        <Route
          path="/wishlist"
          element={
            <Wishlist
              wishlistItems={wishlistItems}
              onRemoveFromWishlist={handleRemoveFromWishlist}
            />
          }
        />
        <Route path="/login" element={<Login onLogin={handleLogin} />} />
        <Route path="/signup" element={<Signup />} />
        <Route path="/home" element={<Home />} />
      </Routes>
    </div>
  </Router>
 );
}

export default App;
```

Navbar.jsx:-

```
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import "./NavBar.css";
import { FontAwesomeIcon } from "@fortawesome/react-fontawesome";
import { faShoppingCart, faHeart } from "@fortawesome/free-solid-svg-icons";

const Navbar = ({
  isLoggedIn,
  username,
  onLogout,
  cartItems,
  wishlistItems,
}) => {
```

```
const [isNavOpen, setIsNavOpen] = useState(false);
const navigate = useNavigate();

const toggleNav = () => {
  setIsNavOpen(!isNavOpen);
};

const handleLogout = () => {
  onLogout();
  navigate("/signup");
};

// Calculate the total number of items in the cart
const cartItemCount = cartItems
  ? cartItems.reduce((total, item) => total + item.quantity, 0)
  : 0;

// Create a Set to store unique product IDs in the wishlist
const uniqueWishlistItemIds = new Set();

// Iterate through the wishlistItems and add their IDs to the Set
wishlistItems.forEach((item) => {
  uniqueWishlistItemIds.add(item.id);
});

// Calculate the total number of unique items in the wishlist
const wishlistItemCount = uniqueWishlistItemIds.size;

return (
  <div className={`navigation-bar ${isNavOpen ? "open" : ""}`}>
    <div className="menu-icon" onClick={toggleNav}>
      <div className="line"></div>
      <div className="line"></div>
      <div className="line"></div>
```

```
</div>
{isNavOpen ? (
  <ul className="nav-list">
    {isLoggedIn ? (
      <>
        <li>
          <Link to="/">Home</Link>
        </li>
        <li>
          <Link to="/products">Product</Link>
        </li>
        <li>
          <Link to="/about">About</Link>
        </li>
        <li>
          <Link to="/contact">Contact</Link>
        </li>
        <li>
          <Link to="/cart">
            <div className="cart-icon">
              <FontAwesomeIcon icon={faShoppingCart} />
              {cartItemCount > 0 && (
                <span className="cart-item-count">
                  {cartItemCount === 1 ? "+1" : `+${cartItemCount}`}
                </span>
              )}
            </div>
          </Link>
        </li>
        <li>
          <Link to="/wishlist">
            <div className="wishlist-icon">
              <FontAwesomeIcon icon={faHeart} />
              {wishlistItemCount > 0 && (
```

```
                <span className="wishlist-item-count">
                  {wishlistItemCount}
                </span>
              )}
            </div>
          </Link>
        </li>
        <li>
          <span>Hello, {username}</span>
        </li>
        <li>
          <button onClick={handleLogout}>Logout</button>
        </li>
      </>
    ) : (
      // Render "Login" and "Signup" links only when not logged in
      <>
        <li>
          <Link to="/home">Home</Link>
        </li>
        <li>
          <Link to="/signup">Signup</Link>
        </li>
        <li>
          <Link to="/login">Login</Link>
        </li>
      </>
    )}
  </ul>
 ) : null}
 </div>
 );
};
```

```
export default Navbar;
```

Signup.jsx:-

```
import React, { useState } from "react";
import "./Signup.css"; // Import the CSS file
import { Link, useNavigate } from "react-router-dom"; // Import useNavigate

const Signup = ({ onSignup }) => {
  const [username, setUsername] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [contact, setContact] = useState("");
  const [type, setType] = useState("");
  const [message, setMessage] = useState("");
  const navigate = useNavigate();

  const handleSignup = async (e) => {
    e.preventDefault();

    try {
      const response = await fetch(
        "http://localhost:5000/backend/user/singup",
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
          },
          body: JSON.stringify({ username, email, password, contact, type }),
        }
      );

      const data = await response.json();

      if (response.ok) {
```

```
      if (data.message === "User Already Exists! Login instead") {
        setMessage(data.message);
      } else if (
        data.message === "Enter name" ||
        data.message === "Enter email" ||
        data.message === "Enter Password" ||
        data.message === "Enter Contact" ||
        data.message === "Enter Type"
      ) {
        setMessage(data.message);
      } else {
        setMessage("Signup successful!");
        onSignup(data.user.username); // Assuming 'user' object is returned from backend
        navigate("/");
      }
    } else {
      setMessage("Signup failed. Please check your information.");
    }
  } catch (error) {
    console.error(error);
  }
};

return (
  <div className="App">
    <div className="form-container">
      <h2>Signup</h2>

      <form onSubmit={handleSignup}>
        <input
          type="text"
          placeholder="Username"
          className="form-input"
          value={username}
```

```
  onChange={(e) => setUsername(e.target.value)}
/>
<input
  type="email"
  placeholder="Email"
  className="form-input"
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
<input
  type="password"
  placeholder="Password"
  className="form-input"
  value={password}
  onChange={(e) => setPassword(e.target.value)}
/>
<input
  type="text"
  placeholder="Contact"
  className="form-input"
  value={contact}
  onChange={(e) => setContact(e.target.value)}
/>
<input
  type="text"
  placeholder="Type"
  className="form-input"
  value={type}
  onChange={(e) => setType(e.target.value)}
/>
<div className="form-button-container">
  <button type="submit" className="form-button">
    Signup
  </button>
```

```
        </div>
      </form>

      <p className="message">{message}</p>
      <p className="login-link">
        Already have an account? <Link to="/login">Login</Link>
      </p>
    </div>
  </div>
 );
};

export default Signup;
```

Login.jsx:-

```
import React, { useState } from "react";
import "./Login.css"; // Import the CSS file
import { Link, useNavigate } from "react-router-dom"; // Import useNavigate

const Login = ({ onLogin }) => {
 const [email, setEmail] = useState("");
 const [password, setPassword] = useState("");
 const [message, setMessage] = useState("");
 const navigate = useNavigate(); // Initialize useNavigate

 const handleLogin = async (e) => {
  e.preventDefault();

  try {
   const response = await fetch("http://localhost:5000/backend/user/login", {
    method: "POST",
    headers: {
     "Content-Type": "application/json",
    },
```

```
      body: JSON.stringify({ email, password }),
    });

    const data = await response.json();

    if (data.message === "Login Successfull") {
      setMessage("Login successful!");
      onLogin(data.username);
      navigate("/"); // Redirect to the homepage
    } else {
      setMessage("Incorrect email or password");
    }
  } catch (error) {
    console.error(error);
  }
};

return (
  <div className="App">
    <div className="form-container">
      <h2>Login</h2>
      <form onSubmit={handleLogin}>
        <input
          type="email"
          placeholder="Email"
          className="form-input"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
        <input
          type="password"
          placeholder="Password"
          className="form-input"
          value={password}
```

```
          onChange={(e) => setPassword(e.target.value)}
        />
        <div className="form-button-container">
          <button type="submit" className="form-button">
            Login
          </button>
        </div>
      </form>
      <p className="message">{message}</p>
      <p className="login-link">
        Don't have an account? <Link to="/signup">Sign up</Link>
      </p>
    </div>
  </div>
 );
};

export default Login;
```

Home.jsx:-

```
import React, { useEffect, useState } from "react";
import "./Home.css";
import { Link } from "react-router-dom";

function Home() {
 const [isLoaded, setIsLoaded] = useState(false);

 useEffect(() => {
  // After the component is mounted, set "loaded" to true after a brief delay
  const timeout = setTimeout(() => {
   setIsLoaded(true);
  }, 500); // Adjust the delay as needed

  // Clear the timeout if the component is unmounted before the delay expires
```

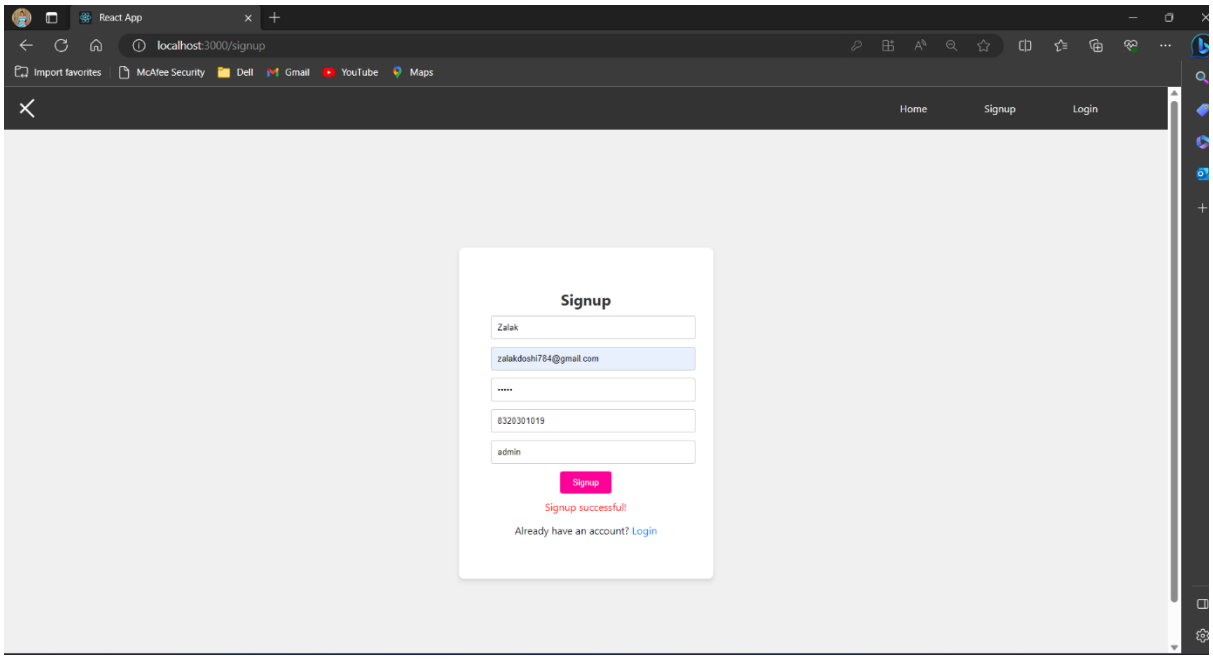
```
    return () => clearTimeout(timeout);
  }, []);

  return (
    <div className={`home ${isLoaded ? "loaded" : ""}`}>
      <div className="hero">
        <h1>Welcome to Our Clothing Store</h1>
        <p>Discover the latest trends in fashion</p>
        <Link to="/products">Shop Now</Link>
      </div>
      <div id="shop" className="product-list">
        {/* Featured product items */}
        {/* Render more product items here */}
      </div>
      <footer className="footer">
        <p>&copy; 2023 Clothing Store. All rights reserved.</p>
      </footer>
    </div>
  );
}

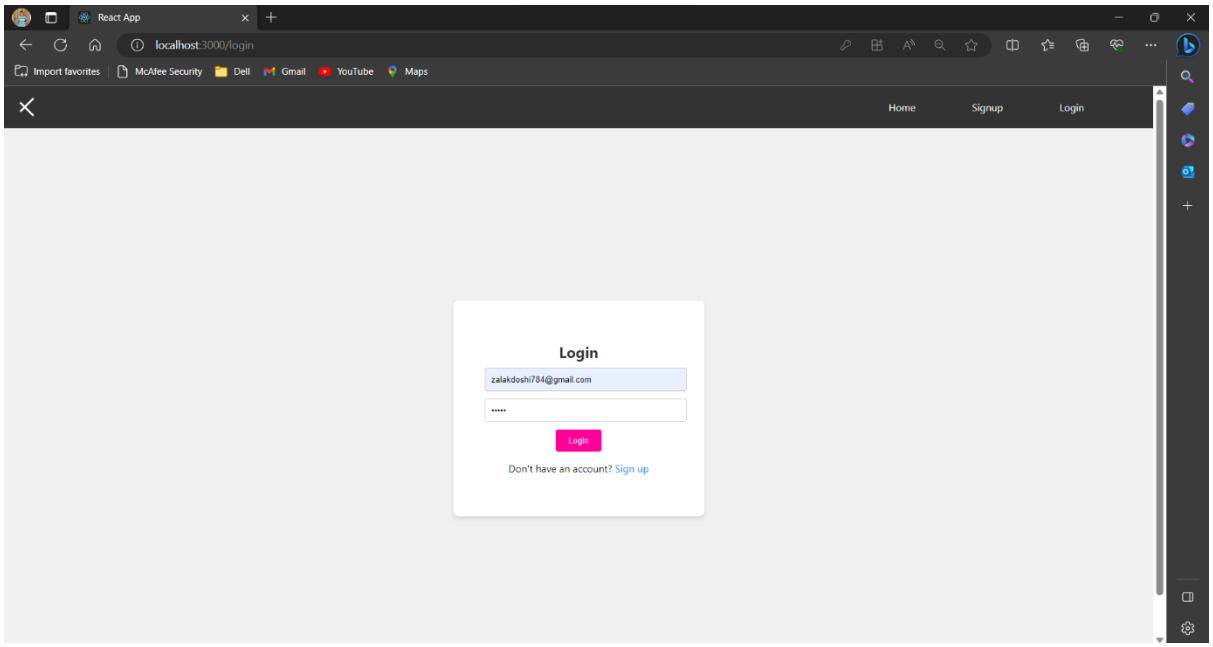
export default Home;
```

Screen shot of various pages of project

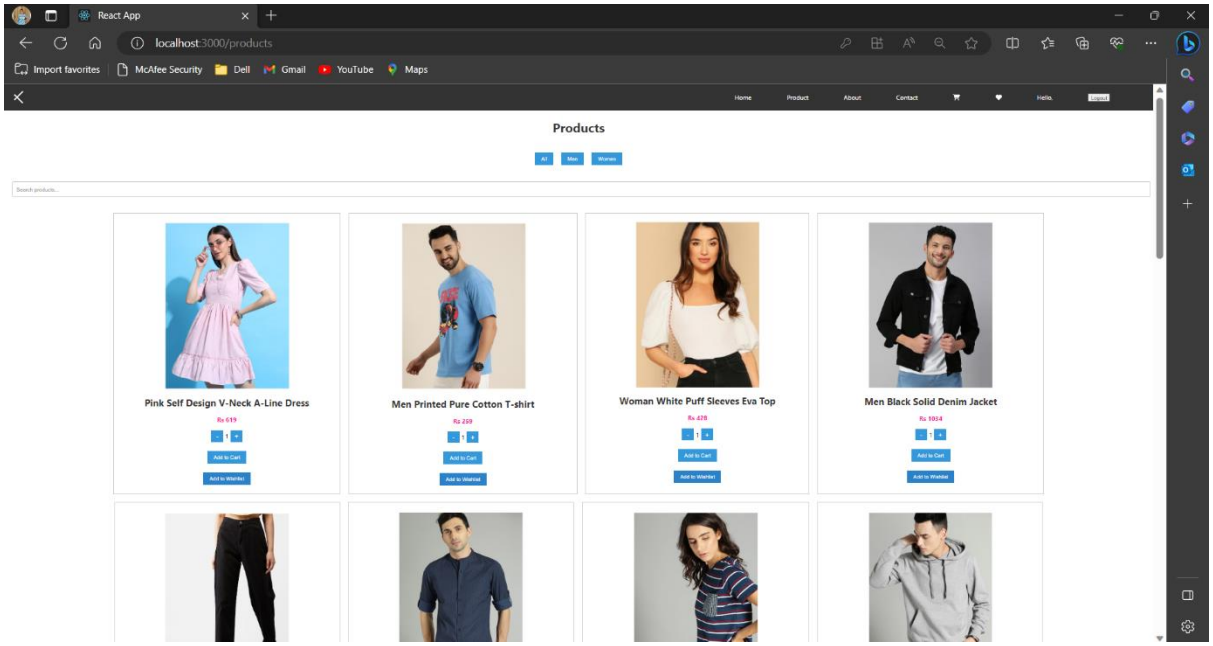
Signup:

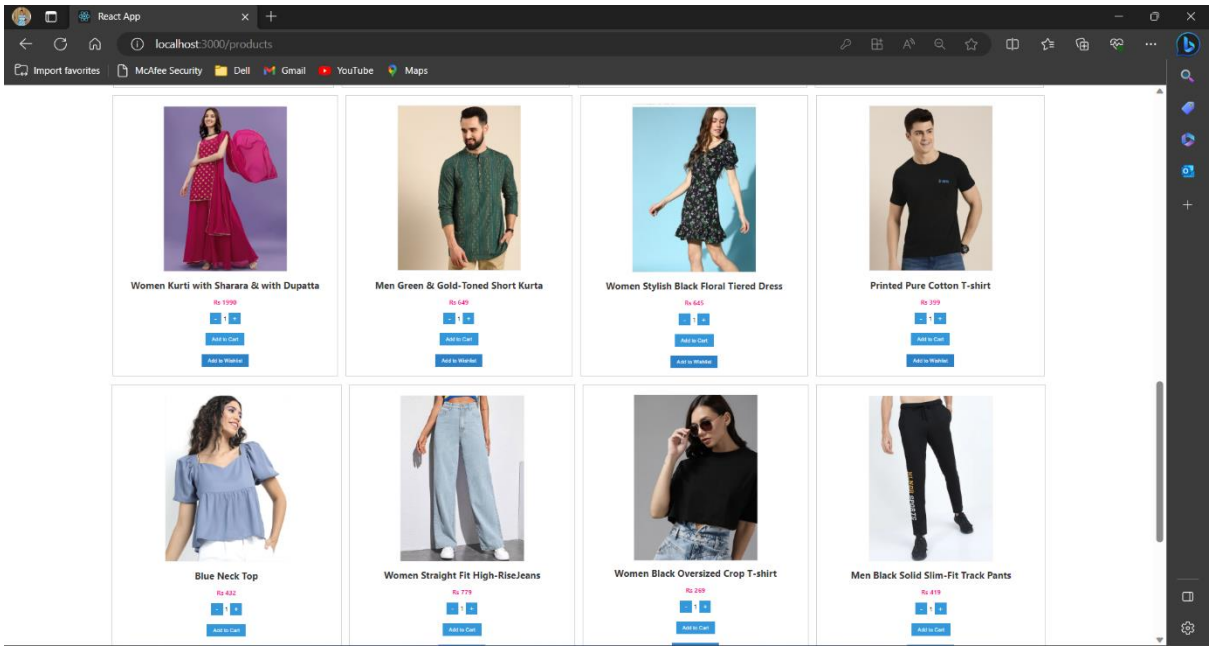


Login:

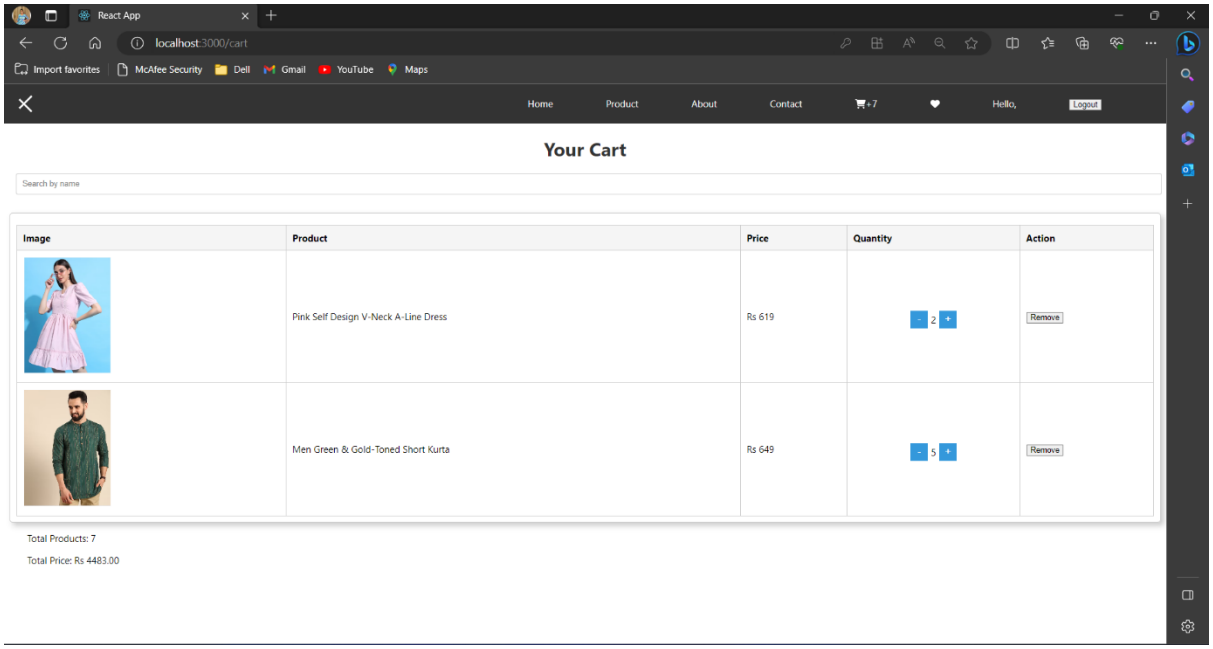


Products:

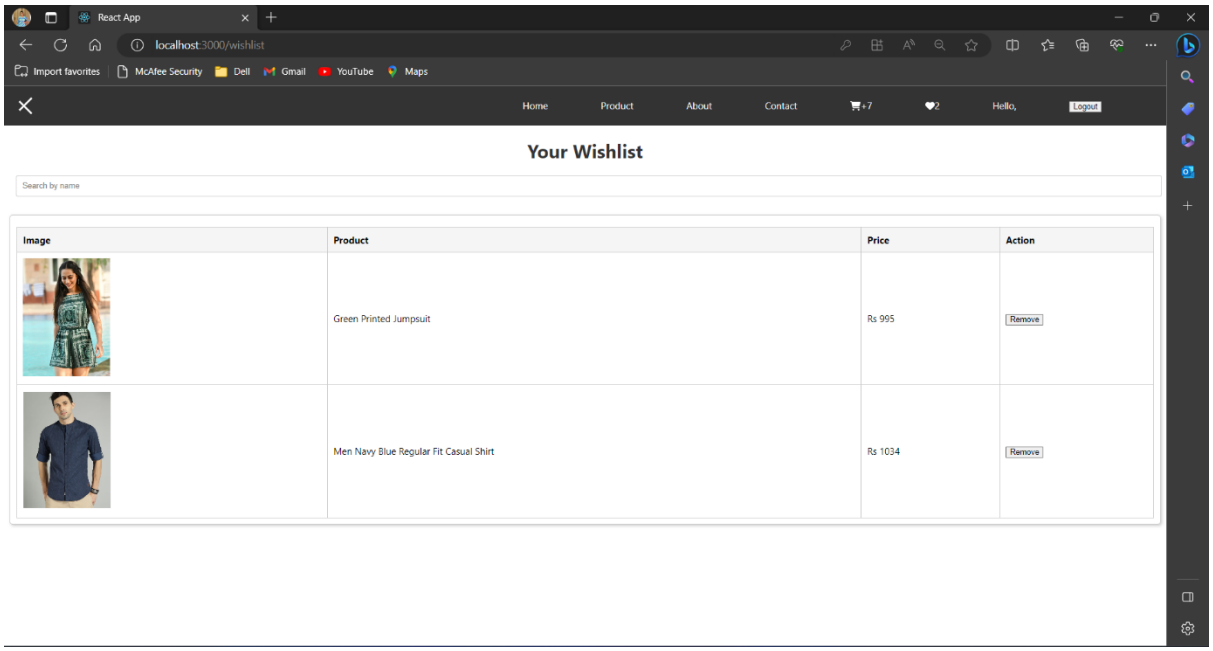

React App
localhost:3000/products
Import favorites
McAfee Security
Dell
Gmail
YouTube
Maps
Home
Product
About
Contact
Products
Pink Self Design V-Neck A-Line Dress
Men Printed Pure Cotton T-shirt
Woman White Puff Sleeves Eva Top
Men Black Solid Denim Jacket


React App
localhost:3000/products
Import favorites
McAfee Security
Dell
Gmail
YouTube
Maps
Women Kurti with Sharara & with Dupatta
Men Green & Gold-Toned Short Kurta
Women Stylish Black Floral Tiered Dress
Printed Pure Cotton T-shirt
Blue Neck Top
Women Straight Fit High-RiseJeans
Women Black Oversized Crop T-shirt
Men Black Solid Slim-Fit Track Pants

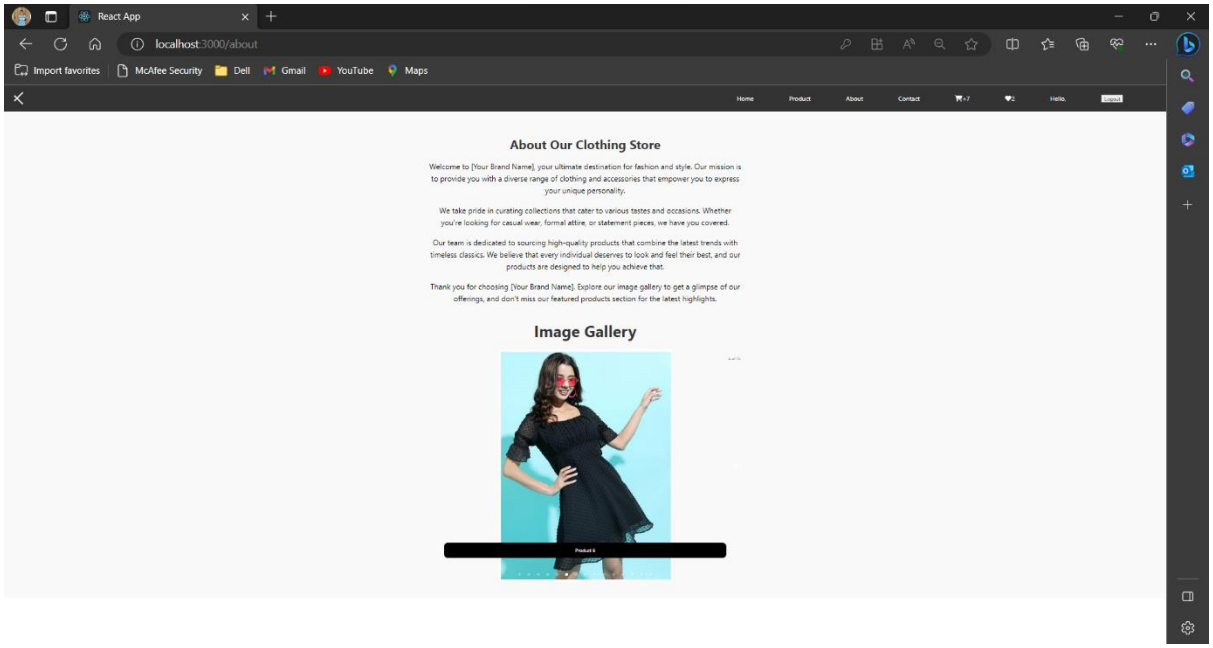
Cart:



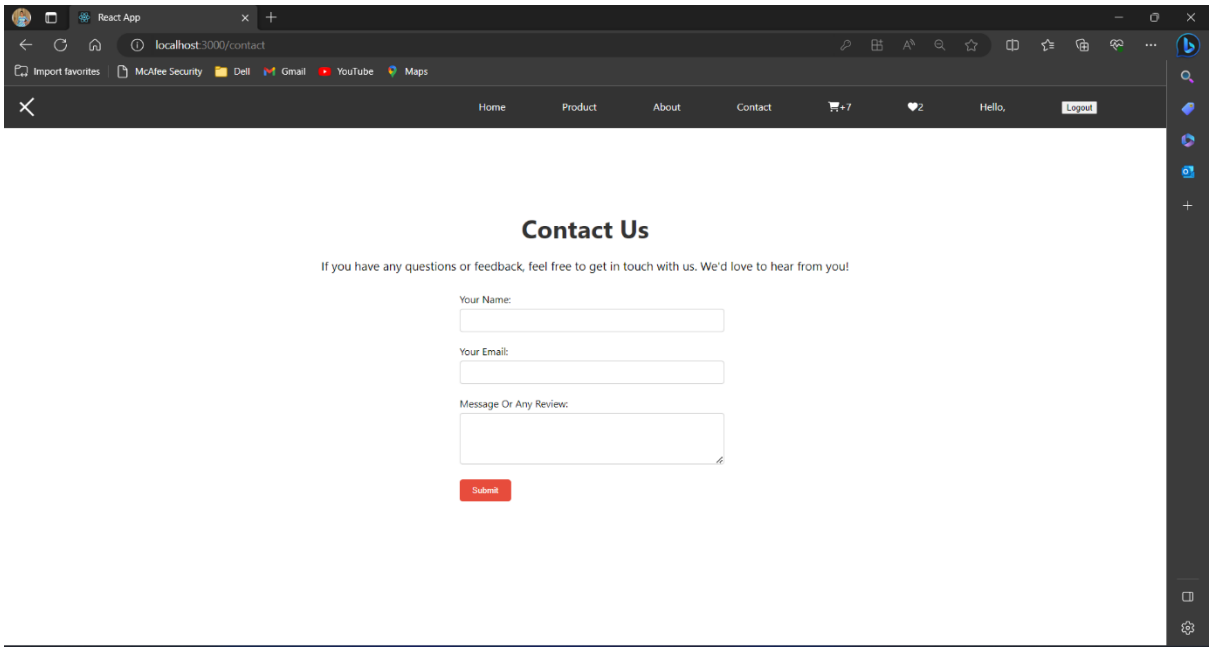
Wishlist:



About:



Contact:



Testing strategies of project

Testing an e-commerce website is crucial to ensure its functionality, security, and user experience meet expectations. Here are some testing strategies and types of testing to consider for an e-commerce website project:

1. **Functional Testing:** Ensure all website functions work correctly, such as registration, product search, and checkout.
2. **Usability Testing:** Assess user-friendliness, navigation, and overall user experience.
3. **Performance Testing:** Check website speed, load handling, and scalability under high traffic.
4. **Security Testing:** Identify and fix vulnerabilities to protect user data and the site from threats.
5. **Compatibility Testing:** Ensure the website works on various browsers and devices.
6. **Content Testing:** Verify the accuracy and timeliness of website content.
7. **SEO Testing:** Optimize the site for search engines to improve visibility.
8. **Regression Testing:** Continuously test for new issues as updates are made.
9. **Accessibility Testing:** Make sure the site is accessible to people with disabilities.
10. **Mobile App Testing:** Test the mobile app version for iOS and Android devices.
11. **User Acceptance Testing:** Validate the website against business requirements and user expectations.

12. **Continuous Testing:** Implement automated testing in the development process.

13. **Scalability Testing:** Ensure the site can handle increased loads without performance issues.

Limitations of the project

E-commerce projects, like any other, have their limitations and challenges. Here are some common limitations and considerations for e-commerce projects:

Technical Challenges:-

1. Scalability: Managing increased traffic during peak times can be challenging.
2. Security: Protecting against cyber attacks, data breaches, and fraud is crucial.
3. Integration: Integrating payment gateways, inventory, and CRM systems can be complex.
4. Performance: Ensuring fast load times and responsiveness across devices requires optimization.

Logistics and Operations:-

1. Inventory Management: Maintaining accurate stock levels can be challenging.
2. Shipping and Logistics: Managing costs, tracking orders, and handling returns efficiently is crucial.
3. Customer Support: Providing effective and timely customer support can be resource-intensive.

User Experience and Conversion:-

1. Cart Abandonment: Many users abandon their shopping carts before completing purchases.
2. User Interface: Designing an intuitive and user-friendly interface is critical.
3. Mobile Responsiveness: Ensuring a seamless mobile experience is essential.

Competition:-

Intense Competition: E-commerce is highly competitive, making it challenging to stand out.

Regulatory and Legal Compliance:-

Legal Requirements: Adhering to data protection, taxation, and consumer protection laws is mandatory.

Payment Processing:-

Complex Payment Processing: Managing secure payment processing can be complex and costly.

Data Privacy and Security:-

Data Protection: Safeguarding customer data and complying with privacy regulations is demanding.

Content and Product Management:-

Content Updates: Maintaining accurate product information and descriptions can be time.

Seasonal Variations:-

Fluctuating Demand: Handling significant fluctuations in demand during seasons or holidays can be challenging.

User Trust:-

Trust Building: Building and maintaining customer trust is vital for e-commerce success.

Conclusion and Future work

In conclusion, the E-Commerce Clothing Website project represents a significant step forward in the world of online fashion retail. It has successfully achieved its primary objectives, which include creating a user-friendly platform for fashion enthusiasts to explore, browse, purchase, and wish list a wide range of clothing products. The project has strived to provide a secure and efficient shopping experience for both men and women, offering a diverse selection of clothing items.

Throughout the development process, a dedicated team of professionals worked tirelessly to ensure the website's functionality, responsiveness, and overall performance. By leveraging a modern technology stack and best practices, they have created a robust and reliable e-commerce platform.

The project's success is reflected in the positive feedback received from users and the steady growth in user engagement and sales. It has established a strong online presence in the fashion retail industry, attracting a loyal customer base.

Future Work:-

While the E-Commerce Clothing Website project has achieved its primary goals, there is always room for improvement and expansion. Here are some areas of future work and potential enhancements:

1. **Mobile Application:** Consider developing a mobile app for the platform to reach a wider audience and provide a more convenient shopping experience.

2. **Personalization:** Implement personalized product recommendations and content to enhance the user's shopping experience based on their preferences and browsing history.

3. **International Expansion**: Explore opportunities to expand the platform's reach to international markets and provide multi-language support.

4. **Integration with Social Media:** Allow users to share their favorite products on social media platforms, enabling viral marketing and increased brand visibility.

5. **AI-Powered Chatbot**: Integrate an AI-powered chatbot for customer support to handle common inquiries and provide assistance 24/7.

6. **Sustainability Initiatives**: Incorporate eco-friendly and sustainable product options, appealing to environmentally conscious consumers.

7. **User-Generated Content**: Encourage users to create and share content related to the products they purchase, such as reviews, photos, and styling tips.

8. **Enhanced Security:** Continuously update and improve security measures to protect user data and transactions from potential threats.

9. **Feedback Mechanism:** Implement a robust feedback system to gather user opinions and suggestions for further improvements.

By addressing these areas of future work, the E-Commerce Clothing Website can continue to evolve and meet the changing needs and expectations of its users, ultimately solidifying its position as a leading online fashion destination.

References

1. Youtube
2. Online website’s
3. ChatGpt
4. Internship